

Full Stack Development with MERN Project

Documentation format

1. Introduction

- **Project Title:** DOCSPOT: Seamless Appointment Booking for Health
- **Team Members: Team Leader :** D Sai Praneetha
- **Team member :** D Manogna
- **Team member :** S Sai Varshitha
- **Team member :** M Vedhanath Reddy
- **Team member :** S Aveen Kumar

2. Project Overview

- **Purpose:** Doc Spot aims to streamline the process of finding and booking medical appointments by providing users with a simple, intuitive platform. The goal is to eliminate the hassle of traditional appointment scheduling, reduce waiting times, and enhance patient experience by offering real-time availability and easy access to healthcare professionals.
- **Features: Seamless Appointment Booking:** Quick and user-friendly interface to book appointments with doctors across specialties.
- **Real-Time Availability:** Live calendar syncing to show up-to-date availability of healthcare providers.
- **Telehealth Integration:** Optional virtual consultation support for remote access to healthcare

3. Architecture

- **Frontend: Framework:** React.js with functional components and React Hooks for state and lifecycle management.
- **UI/UX:**
- Built using component libraries like Material-UI or Tailwind CSS for responsive design.
- Page structure includes Home, Doctor Search, Appointment Booking, Profile Dashboard, and Admin Panel.
- **State Management:**
- **Redux** or **Context API** for managing global state (e.g., user sessions, doctor lists).
- **React Router** for client-side routing and navigation.
- **API Integration:**
- Axios or Fetch API to communicate with backend endpoints.
- Real-time updates through WebSockets (for appointment status, availability).

Backend: Framework: Node.js with Express.js for creating RESTful APIs.

API Structure:

- RESTful endpoints organized by resources: /users, /doctors, /appointments, etc.
- Modular controller-service-repository pattern for scalability.

Authentication & Security:

- JWT-based authentication for users and doctors.
- Role-based access control for Admin, Doctor, and Patient roles.

Database: The MongoDB database includes **Users**, **Doctors**, and **Appointments** collections.

Appointments reference both user and doctor IDs to manage bookings.

Doctors store their available time slots, and users track their appointment history.

Indexes on key fields ensure fast searches and efficient scheduling operations

4. Setup Instructions

- **Prerequisites**
 - [Node.js](#) (v14 or above)
 - [MongoDB](#) (local or cloud instance)
 - **Git**
 - **A code editor (e.g., VS Code)**
- **Installation: Clone the Project Repository:**

Download or clone the project from the GitHub repository to your local machine.
- **Install Dependencies:**

Navigate to both the backend and frontend folders and install the required packages using a package manager like npm.
- **Configure Environment Variables:**

Create .env files in both backend and frontend folders and add necessary environment variables such as database URL, server port, and authentication secrets.
- **Start the Application:**

Run the backend server and frontend client in separate terminal windows.
- **Open the App:**

Once running, access the application in your web browser using the specified local URL (typically <http://localhost:3000>)

5. Folder Structure

Client: The frontend is structured for clarity and scalability:

- **UI Components:** Houses reusable visual elements used across multiple pages.
- **Pages/Views:** Contains the main screens like home, login, register, and appointment dashboard.
- **API Services:** Manages all communication between the frontend and backend.
- **Assets:** Includes images, logos, and CSS styling resources.

Server: The backend is organized to separate logic, routes, and data:

- **Routes:** Defines all RESTful API paths for authentication, appointments, and doctor info.
- **Controllers:** Contains functions that process requests and send responses.
- **Models:** Holds database schema definitions for users, doctors, and appointments.
- **Config:** Stores database connection and environment variable settings.

6. Running the Application

- To run the Doc Spot application locally:
 - **Frontend:** Open a terminal, navigate to the frontend folder, and start the React development server using the appropriate command.
 - **Backend:** Open another terminal, go to the backend folder, and start the Node.js server.
 - Once both servers are running, the frontend will be accessible in your browser (usually at localhost:3000), and the backend API will run on its configured port (commonly localhost:5000).

7. API Documentation

- **User APIs:** Allow registration, login, and user profile retrieval.
- **Doctor APIs:** Provide doctor listings, profile details, and availability updates.
- **Appointment APIs:** Enable booking, viewing, and updating appointment status.
- **Access Control:** Secured with tokens; different roles (doctor/patient) have specific access.
- **Responses:** Return confirmation messages, user/doctor info, and appointment data.

8. Authentication

- **Authentication** is managed using **JSON Web Tokens (JWT)**. When users (patients or doctors) log in, a secure token is generated and sent to the client.
- This token is included in the headers of future requests to verify the user's identity and allow access to protected routes.
- **Authorization** ensures users can only access features based on their roles. For example, only doctors can manage availability, and only patients can book appointments.
- The server checks the token with every request to confirm its validity and verify permissions.

- No session-based login is used; the system is **stateless**, meaning login sessions are handled entirely through token verification.

9. User Interface

- Doc Spot features a clean and intuitive interface designed for easy navigation. Users are welcomed with a home page that links to login, registration, and doctor search. Patients can browse doctors using filters like specialization and location, then book appointments through a simple booking form. Both patients and doctors have personalized dashboards to manage their schedules. The UI is fully responsive, ensuring a smooth experience across devices.

10. Testing

- The testing strategy for Doc Spot focuses on ensuring both functionality and reliability across the frontend and backend.
- **Frontend Testing:** Tools like **Jest** and **React Testing Library** are used to test UI components and user interactions.
- **Backend Testing:** **Mocha**, **Chai**, or **Jest** are used to test API endpoints, database interactions, and business logic.
- **Manual Testing:** Performed to validate user flows such as registration, login, appointment booking, and dashboard access.

11. Screenshots or Demo

- https://drive.google.com/file/d/1xoa3fsUviSYwA5FSq9a2TvemNcaUmAsK/view?usp=drive_link

12. Known Issues

- Some forms lack proper validation and user feedback.
- Appointment slots may be double-booked under high load.
- No email or SMS notifications are integrated yet.
- Minor layout issues on smaller mobile screens.
- Backend error messages are sometimes too generic.

13. Future Enhancements

- Add email/SMS appointment notifications
- Integrate calendar syncing (Google/Outlook)
- Enable online payments and video consultations
- Build an admin dashboard for system management
- Support multiple languages and real-time slot locking

